



IDLookup.AI React Front-End & API Specification for Cursor.ai

Introduction

IDLookup.AI helps users find public information about people and monitor when others search for them. The **frontend** will be a single-page React.js application (HTML, CSS, JS) and will call a back-end API layer implemented by a colleague. The goal of this specification is to define the signup funnel, member experience, API endpoints and authentication model so that Cursor.ai can implement the React front-end consistently and the back-end team can supply the necessary endpoints.

General Design Principles

1. **Clear value proposition** – On the landing/search page and signup screens, clearly state what IDLookup.AI does and why the user should continue. Users decide whether to sign up based on this messaging ¹.
2. **Minimal forms & progressive profiling** – Ask for only essential data in the first step. Additional details can be requested later via progressive profiling to keep friction low ².
3. **Responsive and accessible design** – The interface must work on desktop and mobile. Mobile-optimization is critical because more than half of internet traffic comes from mobile devices ³. Use semantic HTML and ARIA attributes for accessibility.
4. **Consistent dashboard design** – Dashboards should present relevant information at a glance, use big bold numbers and ample white space ⁴, and follow F- and Z-pattern scanning to guide the user's eye ⁵. Group related data into cards/panels and allow users to drill down for details ⁶.
5. **RESTful, versioned API** – All endpoints are under `/api/v1/...`. Versioning from day one prevents breaking changes ⁷. Use nouns for resources, HTTP verbs for intent, and proper HTTP status codes ⁸. Implement pagination, filtering and caching to handle large data sets ⁹.

User Journey & Front-End Pages

1. Search Landing Page

- **Purpose:** Users arrive here from ad units after entering a name. Display the initial search results and encourage them to explore further.
- **Layout:**
- **Search bar** (pre-populated with the query from the ad) plus location field for optional ZIP code.
- **Search results list** – Each result card shows full name, age range, location and a “View Details” button. Provide pagination. Display a disclaimer that results are based on publicly available data.
- **Clear value proposition** – A banner or side panel highlights IDLookup.AI's benefits (finding people, monitoring who searches for you, etc.) and motivates users to click a result ¹.
- **Call to action** – A sign-up button near search results that says “Unlock full report” or similar.
- **API Calls:**

- **GET** `/api/v1/search` – Query parameters: `firstName`, `lastName`, `zip` (optional), `page`, `limit`. Returns an array of search results with basic attributes (id, fullName, ageRange, location). Supports pagination using `limit` and `cursor` parameters ¹⁰.

2. Search Result Detail (Pre-Signup)

- When the user clicks on a search result, show a preview of the report but blur or truncate most details. Indicate that detailed information is available after sign-up.
- A **sign-up modal** or page will open when the user tries to view full details.

3. Sign-Up Funnel

A friction-reduced, progressive sign-up that collects minimal information at each step. Best practices include simple forms, optional social login, password guidance, and clear error messages ¹¹ ¹² ¹³ ¹⁴ ¹⁵.

Step 1 – Create Account

- **Fields:** Name, ZIP code, email and password. Only ask what is necessary to create an account ¹¹. Show password guidelines and allow toggling visibility ¹³ ¹⁴. Offer social login (Google, Apple, Facebook) to reduce friction ¹⁶ ¹².
- **Validation:** Ensure email format, password strength and non-empty fields. Use inline validation with specific error messages ¹⁵.
- **Submit:** POST `/api/v1/signup` with JSON `{fullName, zip, email, password, socialProvider?}`. Return a temporary account record and a verification status.

Step 2 – Email Verification

- After account creation, instruct the user to verify their email. Use a one-click verification link to reduce friction ¹⁷.
- **API:** GET `/api/v1/verify-email?token=...`. Returns success or error.

Step 3 – Payment Information

- Once email is verified, ask for payment details and subscription plan. Explain pricing clearly and emphasize the benefits of membership.
- **Fields:** Payment card details (name, card number, expiration, CVV) and billing address. Use secure components that handle card data via a PCI-compliant provider (e.g., Stripe). Provide a summary of charges.
- **API:** POST `/api/v1/subscribe` with plan, payment token and billing info. Returns subscription confirmation.

Progressive Profiling & Onboarding

- After subscription, collect optional profile attributes (date of birth, additional addresses, previous names) gradually over time; do not overwhelm users with long forms ¹⁸.
- Provide a guided onboarding tour that explains how to use search, set up alerts and view who is searching for them ¹⁹.

4. Login Page

- **Fields:** Email and password. Provide “Show password” toggle and “Forgot password” link. Offer social login.
- **API:** POST `/api/v1/login` returns an access token and refresh token.
- **Error handling:** Show specific messages for invalid credentials or unverified accounts.

5. Member Dashboard

After successful login/subscription, users enter the member area. The dashboard contains multiple views, accessible via a navigation sidebar or top bar. Use cards and panels to present data clearly with key metrics highlighted ⁴.

Dashboard Home

- **Who searched for you** – A card displaying the number of times the user’s profile was viewed this month and a list of the most recent searchers (obfuscated names until user clicks). Provide filters (last 24 hours, last 7 days, etc.).
- **Recent activity** – Show a timeline of user actions (searches performed, alerts triggered).
- **Alerts summary** – Display the number of active search alerts and link to manage them.
- **Subscription status** – Show current plan, renewal date and a quick link to update payment.

Personal Profile Page

- Shows personal information gathered from the user and public records. Allow editing of self-provided data (name, email, phone, address). Display public info but restrict editing (since it comes from external databases). Provide a link to request corrections or removals.
- **API:**
- GET `/api/v1/me` – Returns user profile and membership status.
- PUT `/api/v1/me` – Updates editable fields after validation.

People Search Page (Member)

- A more powerful search interface than the landing page.
- **Fields:** Name, phone number, email address, address and optional filters (state, age range). Provide auto-complete suggestions. Include search history table.
- **Results:** Display a table or cards with names, locations and age. Clicking a result opens a full report with contact info, address history, relatives and associated records.
- **API:**
- GET `/api/v1/search` – With more filters.
- GET `/api/v1/people/:id` – Returns full report for selected person.

Who Is Searching Me

- A dedicated view that lists recent instances when other users or visitors searched for this member’s name. Display the searcher’s approximate location (city/state) and time of search. Provide filters and the option to drill down into each event.

- **API:** GET `/api/v1/searches/me` – Returns a paginated list of search events with `timestamp`, `location`, `searcherId` (anonymized) and searcher membership level.

Search Alerts Management

- Users can create alerts on specific names or personal information. When a match occurs, the system sends an email or in-app notification.
- **UI:** Provide a list of existing alerts with status (active/inactive) and triggers. Offer buttons to add, edit or delete alerts.
- **API:**
 - GET `/api/v1/alerts` – List current alerts.
 - POST `/api/v1/alerts` – Create an alert (payload: search criteria, frequency, notification channels).
 - PUT `/api/v1/alerts/:id` – Update alert settings.
 - DELETE `/api/v1/alerts/:id` – Remove an alert.

Account & Billing Management

- A page to view subscription details, billing history and update payment method. Allow the user to upgrade, downgrade or cancel the plan. Provide an option to download invoices.
- **API:**
 - GET `/api/v1/subscription` – Returns subscription and billing details.
 - PUT `/api/v1/subscription` – Update plan.
 - DELETE `/api/v1/subscription` – Cancel membership (with confirmation and effective date).
 - GET `/api/v1/invoices` – Returns list of invoices with download URLs.

6. Notifications

- In-app notifications inform users when someone searches for them or when an alert is triggered. A bell icon opens a notification panel. Notifications can be marked as read or archived.
- **API:**
 - GET `/api/v1/notifications` – List notifications.
 - PUT `/api/v1/notifications/:id/read` – Mark as read.
 - DELETE `/api/v1/notifications/:id` – Remove.

7. Settings & Security

- Users can change password, manage multi-factor authentication (MFA) and control privacy settings (e.g., opt out of being searchable). Provide toggles and explanatory text.
- **API:**
 - POST `/api/v1/auth/change-password` – Change password.
 - POST `/api/v1/auth/mfa/enable` and `/api/v1/auth/mfa/disable`.
 - PUT `/api/v1/privacy` – Update privacy preferences.

Page Categories & Navigation

IDLookup.AI serves three distinct groups of users, each with its own set of pages and navigation menus:

1. Sales Pages (public)

These pages are accessible without authentication:

- **Home, About & Contact** – Explain what the service does, highlight the value proposition and provide a contact form for inquiries. The home page is the entry point for organic visitors and should feature a clear call-to-action ¹.
- **Search landing pages** – When a user arrives from an ad campaign with a query (name, email, address or phone), they see a pre-populated search bar and a list of public search results. They can click a result to view a truncated preview.
- **Logged-out search results & detail preview** – Provide enough information to interest the user but blur sensitive data until they sign up.
- **Sign up / login** – A progressive sign-up funnel with account creation, email verification and payment capture, as described under “Sign-Up Funnel.” A separate login page allows returning users to authenticate.

Navigation: Sales pages share a simple top navigation bar that includes the logo on the left and links to *Home*, *About*, *Contact*, and *Sign Up / Login*. On search results pages, show a secondary bar with the search form and a button to sign up or view membership benefits. Keep navigation minimal to focus the user on conversion ¹.

2. Member Pages (authenticated)

Once a user has signed up and logged in, they access member-only pages:

- **Dashboard** – The home screen showing “Who searched for you,” recent activity, alerts summary and subscription status as described in the “Member Dashboard” section.
- **Profile** – Allows the member to view and edit their information.
- **People search & results** – A more powerful search interface with filters and the ability to view full reports.
- **Search result detail/report** – Displays detailed information for a person, including contact info, relatives and associated records.
- **Who is searching me** – A list of search events where others looked up the member.
- **Alerts** – Manage search alerts.
- **Account/Billing** – Manage subscription and payment information.
- **Settings / Security** – Manage password, MFA and privacy settings.
- **Logout** – End the session.

Navigation: Member pages use a persistent navigation bar (e.g., sidebar or top bar) tailored to authenticated users. It contains links to *Dashboard*, *Profile*, *Search*, *Who Is Searching Me*, *Alerts*, *Account* and *Logout*. The nav highlights the current section and collapses or hides items on mobile for responsiveness. Role-based rendering ensures members do not see admin links ²⁰.

3. Admin Pages (internal)

Administrators and customer-service representatives have additional pages to manage the platform:

- **Users** – View and search all user accounts. Provides list with filter/search, ability to suspend or delete accounts.
- **User detail & history** – Drill-down page showing a user's profile, history of searches, login sessions, alerts and subscriptions.
- **Sessions** – List active and past sessions for auditing and security.
- **Purchases** – View all purchases/subscriptions. A *purchase detail* page shows payment info and invoice status.
- **Data removal** – Manage requests from users to remove their personal data (for GDPR/CCPA compliance).
- **Analytics** – Visualize site metrics, search volumes, conversions, churn and other KPIs. Use charts and dashboards designed per F- and Z-pattern scanning ⁵ ⁴.
- **CS rep management** – Manage customer-service agents, assign roles and monitor performance.

Navigation: Admin pages display an admin-specific navigation bar, typically a sidebar with links to *Users*, *Sessions*, *Purchases*, *Data Removal*, *Analytics* and *CS Rep Management*. The nav may be collapsible to maximize workspace. Only users with the `admin` role should access these routes; ensure proper role checks in the front-end and API.

Folder Structure & Routing

Organize the pages in the React project under `src/pages` with subfolders for each group:

```
src/pages/  
  sales/  
    HomePage.jsx  
    AboutPage.jsx  
    ContactPage.jsx  
    LandingPage.jsx  
    SearchResultsPage.jsx  
    SearchDetailPreviewPage.jsx  
    SignupPage.jsx  
    LoginPage.jsx  
    PaymentPage.jsx  
  member/  
    DashboardHome.jsx  
    ProfilePage.jsx  
    SearchPage.jsx  
    SearchResultsPage.jsx  
    SearchResultDetailPage.jsx  
    WhoIsSearchingPage.jsx  
    AlertsPage.jsx  
    AccountPage.jsx  
    SettingsPage.jsx
```

```
LogoutPage.jsx
admin/
  UsersPage.jsx
  UserDetailPage.jsx
  SessionsPage.jsx
  PurchasesPage.jsx
  PurchaseDetailPage.jsx
  DataRemovalPage.jsx
  AnalyticsPage.jsx
  CsRepManagementPage.jsx
```

Implement separate navigation components (`SalesNav` , `MemberNav` , `AdminNav`) and render them based on the current route and user role. The `Header` component can decide which nav to show by inspecting authentication state and role in context.

API Specification

General Conventions

- **Base URL:** `/api/v1`.
- **Authentication:** Most endpoints require a valid access token in the `Authorization: Bearer <token>` header. Endpoints such as `/signup` , `/login` , `/verify-email` and `/search` (public version) are unauthenticated. Admin endpoints under `/admin/*` additionally require the token to include the `admin` role claim. The server should reject requests without sufficient privileges and return `403 Forbidden` ²⁰ .
- **Responses:** Use standard status codes. On errors, return a structured error object with code, message and details ⁸ . Example:

```
{
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "email is required",
    "details": [ {"field": "email", "issue": "missing"} ],
    "traceId": "abc123"
  }
}
```

- **Pagination & Filtering:** Endpoints returning collections support `limit` (max items per page), `cursor` (opaque string for next page) and `fields` parameters to select fields ²¹ .

Key Endpoints

Endpoint	Method & Payload	Description
<code>/signup</code>	POST <code>{fullName, zip, email, password, socialProvider?}</code>	Create a new user account. Returns verification instructions and a temporary access token.
<code>/verify-email</code>	GET <code>?token=...</code>	Verify a user's email address. Returns success message and activates account.
<code>/login</code>	POST <code>{email, password}</code> or social token	Log in and receive <code>accessToken</code> (short-lived) and <code>refreshToken</code> (long-lived).
<code>/refresh-token</code>	POST <code>{refreshToken}</code>	Obtain a new access token when the current one expires.
<code>/logout</code>	POST (no body)	Invalidate the refresh token on the server and clear cookies.
<code>/search</code>	GET <code>?firstName&lastName&zip&page&limit</code>	Public search endpoint for landing page. Returns basic result list.
<code>/people/{id}</code>	GET	Returns full report for a person; requires authentication.
<code>/me</code>	GET	Get authenticated user profile and subscription status.
<code>/me</code>	PUT <code>{fullName?, zip?, email?}</code>	Update editable profile fields.
<code>/dashboard</code>	GET	Returns aggregated stats for the dashboard home (search counts, active alerts, etc.).
<code>/searches/me</code>	GET	Returns list of events where others searched for the user.

Endpoint	Method & Payload	Description
/alerts	GET	List all alerts for the user.
/alerts	POST {criteria, frequency, channels}	Create a new search alert.
/alerts/{id}	PUT {criteria?, frequency?, channels?, status?}	Update an existing alert.
/alerts/{id}	DELETE	Delete an alert.
/subscription	GET	Get current subscription and billing details.
/subscription	PUT {plan, paymentToken}	Change plan or update payment method.
/subscription	DELETE	Cancel subscription.
/invoices	GET	List invoices. Optional ?download=true returns PDF download links.
/notifications	GET	List notifications.
/notifications/{id}/read	PUT	Mark a notification as read.
/notifications/{id}	DELETE	Delete notification.
/privacy	PUT {searchable: boolean}	Update privacy settings.
/auth/change-password	POST {currentPassword, newPassword}	Change the user's password.
/auth/mfa/enable	POST {method}	Enable MFA (e.g., TOTP).
/auth/mfa/disable	POST	Disable MFA.
/admin/users	GET	List all user accounts; admin only. Supports filters and pagination.
/admin/users/{id}	GET	Retrieve details for a user, including profile, history and subscriptions.
/admin/users/{id}/suspend	POST {status}	Suspend or reactivate a user account.

Endpoint	Method & Payload	Description
<code>/admin/sessions</code>	GET	List all active and past sessions for auditing; supports filtering by date range and userId.
<code>/admin/purchases</code>	GET	List all purchases/subscriptions with pagination.
<code>/admin/purchases/{id}</code>	GET	Get details of a specific purchase or subscription.
<code>/admin/purchases/{id}/refund</code>	POST <code>{reason}</code>	Process a refund or cancel a purchase; returns updated purchase status.
<code>/admin/data-removal</code>	GET	List pending data removal requests.
<code>/admin/data-removal/{id}/approve</code>	POST	Approve a data removal request and trigger deletion.
<code>/admin/data-removal/{id}/reject</code>	POST <code>{reason}</code>	Reject a data removal request with explanation.
<code>/admin/analytics</code>	GET	Return aggregated metrics (total searches, conversions, revenue, churn, etc.).
<code>/admin/cs-reps</code>	GET	List customer-service representatives and their roles.
<code>/admin/cs-reps</code>	POST <code>{name, email, role}</code>	Create a new customer-service representative account.
<code>/admin/cs-reps/{id}</code>	PUT <code>{role?, status?}</code>	Update the role of a CS representative or suspend/activate their account.

Authentication Model

IDLookup.AI will use **token-based authentication with JSON Web Tokens (JWTs)**. Upon successful login, the server issues an **access token** with a short expiration (e.g., 15–30 minutes) and a **refresh token** with a longer expiration (e.g., 7 days). The React application sends the access token in the `Authorization`

header for each request. When the access token expires, it calls `/refresh-token` with the refresh token to obtain a new access token.

Token Storage & Security Considerations

- **Preferred method – HttpOnly cookies:** Store the refresh token in an `HttpOnly` cookie with the `Secure` flag and optionally `SameSite=strict` to prevent XSS and CSRF attacks ²². Cookies adhere to the same-origin policy and support setting expiration dates, making them well-suited for long-lived tokens ²³.
- **Access token storage:** Save the short-lived access token in browser memory (e.g., React state or a context provider). This avoids persistent storage and reduces the risk of token theft if a third-party script is compromised ²⁴. When the page is refreshed, use the refresh token cookie to obtain a new access token.
- **Alternative – sessionStorage/localStorage:** If you choose to store tokens in web storage for convenience, consider encrypting them and setting short validity periods ²⁵. However, storing JWTs in localStorage exposes them to XSS attacks ²⁶. Using secure cookies is preferred for persistence ²⁷.
- **Short validity & refresh:** Set a short lifespan for access tokens and implement a refresh mechanism to reduce the window of opportunity for attackers ²⁸. Use refresh tokens that can be rotated and revoked if suspicious activity is detected.
- **Role-based access control:** Include user roles in the JWT claims. On the client, use conditional rendering to show or hide features depending on role (e.g., admin vs. regular user) ²⁰.
- **Email verification:** Require users to verify their email addresses before granting full access. This step helps ensure a higher-quality user base and reduces spam signups ¹⁷.

Form Handling & Validation

- Use controlled React components for all forms and handle state via React Hooks. Provide inline validation and helpful error messages. Do not submit the form until validation passes.
- Display password instructions near the password field and provide real-time feedback on strength ¹³. Include a show/hide password toggle ¹⁴.
- After a server error (e.g., email already registered), display the server's error message using the standardized error envelope.

State Management & Routing

- Use **React Router** for client-side routing. Define routes for `/`, `/result/:id`, `/signup`, `/login`, `/dashboard`, `/profile`, `/search`, `/alerts`, `/account`, `/settings` etc.
- Use **React Context** or a state management library like Redux or Zustand to store authentication state (tokens) and user data across the application.
- Implement a **ProtectedRoute** component that checks if a user is authenticated before rendering a route. If not, redirect to `/login`.

Component Structure (Suggested)

- `App.jsx` – root component with route definitions and global providers.

- `components/` – reusable UI components such as `Header`, `Footer`, `SearchBar`, `ResultCard`, `Modal`, `NotificationBell`, and navigation components (`SalesNav`, `MemberNav`, `AdminNav`). The `Header` chooses which nav component to render based on the user's authentication state and role.
- `pages/` – page-level components organized by user type. Under `pages/sales`, include `HomePage`, `AboutPage`, `ContactPage`, `LandingPage`, `SearchResultsPage`, `SearchDetailPreviewPage`, `SignupPage`, `LoginPage` and `PaymentPage`. Under `pages/member`, include `DashboardHome`, `ProfilePage`, `SearchPage`, `SearchResultsPage`, `SearchResultDetailPage`, `WhoIsSearchingPage`, `AlertsPage`, `AccountPage`, `SettingsPage` and `LogoutPage`. Under `pages/admin`, include `UsersPage`, `UserDetailPage`, `SessionsPage`, `PurchasesPage`, `PurchaseDetailPage`, `DataRemovalPage`, `AnalyticsPage` and `CsRepManagementPage`.
- `api/` – wrapper around `fetch` or `axios` to handle requests, inject tokens and handle refresh.
- `context/AuthContext.jsx` – provides authentication state, login/logout functions and refresh logic.

Error Handling & Feedback

- Show loading indicators during network calls.
- Use toast notifications or banners to display success (e.g., “Alert created”) or error messages (e.g., “Payment failed”).
- Catch network errors and unexpected exceptions and show a generic error page with contact support information.

Security & Privacy Measures

- Use HTTPS for all network communication.
- Sanitize user input on the client and server. Escape HTML to prevent XSS.
- Implement CSRF protection when using cookies for authentication (the `SameSite` flag and anti-CSRF tokens).
- Comply with data protection regulations (e.g., GDPR/CCPA): provide a privacy policy, allow users to request data removal and implement appropriate consent flows.

Conclusion

This specification describes the complete user journey, UI components, API contracts and authentication model for IDLookup.AI's React front-end. By following the best practices—minimizing sign-up friction ², using progressive profiling ¹⁸, designing dashboards that highlight key data ⁴, implementing RESTful versioned APIs ⁷ and storing tokens securely ²²—the IDLookup.AI platform will offer a smooth, secure and user-friendly experience. Cursor.ai can implement these guidelines to build the front-end while coordinating with the back-end team to expose the described endpoints.

¹ ² ³ ¹⁶ ¹⁷ ¹⁸ ¹⁹ 12 Best Practices for Frictionless Sign Up Flows (With Examples)
<https://www.pingidentity.com/en/resources/blog/post/frictionless-signup.html>

4 5 6 Dashboard Design: best practices and examples - Justinmind

<https://www.justinmind.com/ui-design/dashboard-design-best-practices-ux>

7 8 9 10 21 RESTful API Design Guide: Principles & Best Practices

<https://strapi.io/blog/restful-api-design-guide-principles-best-practices>

11 12 13 14 15 5 Best Practices for the Sign-up Flow (with examples!) | by Anastasia Prokhorova |

Prototypr

<https://blog.prototypr.io/5-best-practices-for-the-sign-up-flow-with-examples-f55832edc8a3>

20 A Practical Guide to Authentication and Authorization in React

<https://www.ignek.com/blog/authentication-and-authorization-in-react/>

22 23 24 25 26 27 28 LocalStorage vs Cookies: the best-practice guide to storing JWT tokens securely in your front-end - Cyber Chief

<https://www.cyberchief.ai/2023/05/secure-jwt-token-storage.html>